

NEUROSCAN

Backend Migration Guide

Vibe-coded → Production-Grade Architecture

Architecture Vertical Slice + Service Layer	RBAC Pattern Dependency-Injected Guards
Real-Time Server-Sent Events (SSE)	Phases 5 Sequential Phases

1. Critical Bugs to Fix Before Anything Else

These are not refactoring items — they are active defects that will cause runtime failures, security vulnerabilities, or data corruption. Address them before touching anything else.

● BUG 1 — Hardcoded SECRET_KEY

app/core/security.py has SECRET_KEY = "super_secret_random_string_at_least_32_chars" as a fallback. If the .env file is not loaded before this module imports, this literal value silently becomes your JWT signing key. Every token signed by it is compromised.

● BUG 2 — Enum Mismatch on CaseStatus

models.py defines CaseStatusEnum as PENDING/COMPLETED. But cases.py router references CaseStatusEnum.PROCESSING and CaseStatusEnum.REVIEWED which do not exist. This throws an AttributeError at runtime when the stats endpoint is hit.

● BUG 3 — Route Ordering: /discover Unreachable

In workspaces.py, the route GET /workspaces/discover is defined AFTER GET /workspaces/{workspace_id}. FastAPI matches routes in registration order, so 'discover' is treated as a workspace_id string. The endpoint is permanently unreachable.

● BUG 4 — Shadow Import: Dead Pydantic User Model

app/models/user.py defines a Pydantic class named User. app/models/models.py defines an SQLAlchemy class also named User. Any file that imports from the wrong path gets the wrong class with no error — silent data bugs.

● BUG 5 — Singleton Async Session

session_manager.py implements a singleton AsyncSession shared across all requests. SQLAlchemy sessions are NOT thread-safe. Under concurrent load, two requests will share a session, corrupt each other's transactions, and produce data integrity failures.

⚠ BUG 6 — require_role() Is Never Used

deps.py has a correctly-designed require_role() dependency factory. Zero routers actually import or use it. Every router does ad-hoc membership queries inline — duplicated, inconsistent, and easy to forget on new endpoints.

⚠ BUG 7 — Files Saved to Local Disk Without Scoping

files.py writes uploads directly to a local ./uploads/ directory. There is no workspace scoping (any user can potentially read any file), no MIME type validation, and no virus scanning. This is not HIPAA-compliant.

2. Target Architecture

The new architecture applies three patterns simultaneously. Understanding all three before writing code is critical.

2.1 Vertical Slice Architecture (Domain Organisation)

Each business domain (auth, workspaces, patients, cases, invitations) is a self-contained folder containing its own `router.py`, `service.py`, and `schemas.py`. Nothing leaks across domain boundaries except shared models and dependencies.

- **Router:** Parses HTTP request, calls service, returns response. Zero business logic.
- **Service:** All business logic lives here. Receives typed inputs, returns typed outputs.
- **Schemas:** Domain-specific Pydantic request/response models. No mixing of concerns.

2.2 Dependency-Injected RBAC

Every protected endpoint receives a `WorkspaceContext` dataclass injected by FastAPI's DI system. The context is pre-validated — workspace membership is confirmed, role is checked, and the caller's `member_id` is resolved. The router never queries `WorkspaceMember` directly.

- If the user is not a member → 403 is raised before the router function body executes.
- If the user's role is insufficient → 403 is raised before the router function body executes.
- Adding a new protected endpoint = one line: `Depends(require_workspace_role(...))`

2.3 SSE Event Bus for Real-Time Updates

An in-process `EventBus` singleton manages per-workspace subscriber queues. Services publish events after DB commits. A persistent SSE endpoint streams events to all connected clients. The frontend uses a `useWorkspaceEvents()` hook that invalidates React Query caches on arrival.

2.4 Complete Target File Structure

```
auth-service/
├── main.py                # App factory + lifespan
├── pyproject.toml         # Replaces requirements.txt
├── alembic.ini            # DB migrations config
├── app/
│   ├── core/
│   │   ├── config.py      # Pydantic BaseSettings (all env vars)
│   │   ├── security.py    # JWT + password hashing (no hardcoding)
│   │   ├── event_bus.py   # SSE pub/sub singleton
│   │   ├── exceptions.py  # Custom HTTP exceptions
│   │   └── middleware.py  # CORS, request-id injection
│   ├── db/
│   │   ├── base.py        # Declarative Base (no engine here)
│   │   ├── session.py     # Engine + per-request SessionLocal
│   │   ├── migrations/    # Alembic migration scripts
│   │   └── env.py
│   └── dependencies/
│       ├── auth.py        # get_current_user
│       ├── db.py          # get_db
│       └── rbac.py        # require_workspace_role(), WorkspaceContext
```

```

├── models/
│   ├── __init__.py           # Re-exports all models for Alembic
│   ├── user.py              # User + PendingUser SQLAlchemy models
│   ├── workspace.py         # Workspace + WorkspaceMember models
│   ├── patient.py           # Patient model
│   ├── case.py              # Case model + CaseStatusEnum
│   └── invitation.py        # Invitation + JoinRequest models
├── domains/
│   ├── auth/
│   │   ├── router.py
│   │   ├── service.py
│   │   └── schemas.py
│   ├── workspaces/
│   │   ├── router.py
│   │   ├── service.py
│   │   └── schemas.py
│   ├── patients/
│   │   ├── router.py
│   │   ├── service.py
│   │   └── schemas.py
│   ├── cases/
│   │   ├── router.py
│   │   ├── service.py
│   │   └── schemas.py
│   └── invitations/
│       ├── router.py
│       ├── service.py
│       └── schemas.py

```

3. Migration Phases — Step by Step

Execute phases strictly in order. Each phase builds on the previous. Do not skip ahead.

PHASE 1

Foundation — Configuration & Models

Fix the hardcoded secret. Split the monolithic models file. Kill dead code.

Phase 1 — Step Checklist

Step	Action	File/Location	Risk
1.1	Install python-dotenv and pydantic-settings if not present	<code>requirements.txt</code>	LOW
1.2	Create <code>app/core/config.py</code> — typed Pydantic Settings class	<code>app/core/config.py</code>	MED
1.3	Rewrite <code>app/core/security.py</code> to import <code>SECRET_KEY</code> from config	<code>app/core/security.py</code>	HIGH
1.4	Create <code>app/db/base.py</code> — move Base here, nothing else	<code>app/db/base.py</code>	LOW
1.5	Rewrite <code>app/db/session.py</code> — import Base from <code>base.py</code> , remove singleton manager	<code>app/db/session.py</code>	MED
1.6	DELETE <code>app/db/session_manager.py</code> — it is the dangerous singleton	DELETE	HIGH
1.7	Fix <code>CaseStatusEnum</code> : change COMPLETED to REVIEWED, add PROCESSING	<code>models.py</code> (temp)	HIGH
1.8	Split <code>models.py</code> into per-domain files under <code>app/models/</code>	<code>app/models/*.py</code>	MED
1.9	DELETE <code>app/models/user.py</code> — the dead Pydantic shadow file	DELETE	HIGH
1.10	Create <code>app/models/__init__.py</code> re-exporting all models	<code>app/models/__init__.py</code>	LOW
1.11	Test: <code>python -c 'from app.models import User, Case'</code> must not throw	terminal	LOW

Phase 1.2 — config.py (Full Implementation)

```
# app/core/config.py
from functools import lru_cache
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
```

```

model_config = SettingsConfigDict(
    env_file=".env", env_file_encoding="utf-8", case_sensitive=True
)

# JWT
SECRET_KEY: str # REQUIRED - no default
ALGORITHM: str = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES: int = 30

# Database
SUPABASE_DB_USER: str = ""
SUPABASE_DB_PASSWORD: str = ""
SUPABASE_DB_HOST: str = ""
SUPABASE_DB_PORT: str = "5432"
SUPABASE_DB_NAME: str = "postgres"

# App
ENVIRONMENT: str = "development"
CORS_ORIGINS: list[str] = ["http://localhost:3000"]

@property
def DATABASE_URL(self) -> str:
    if self.SUPABASE_DB_HOST:
        return (
            f"postgresql+psycopg2://{self.SUPABASE_DB_USER}:{self.SUPABASE_DB_PASSWORD}"
            f"@{self.SUPABASE_DB_HOST}:{self.SUPABASE_DB_PORT}/{self.SUPABASE_DB_NAME}"
            "?sslmode=require"
        )
    return "sqlite:///./dev.db"

@lru_cache(maxsize=1)
def get_settings() -> Settings:
    return Settings()

# Single import point for the whole app
settings = get_settings()

```

Phase 1.3 — security.py (Fixed)

```

# app/core/security.py
from datetime import datetime, timedelta
from jose import jwt
from passlib.context import CryptContext
from app.core.config import settings # ← No more hardcoded values

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)

```

```
def create_access_token(data: dict, expires_delta: timedelta | None = None) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or
timedelta(minutes=settings.ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode['sub'] = str(to_encode['sub']) # Always stringify user_id
    to_encode['exp'] = expire
    return jwt.encode(to_encode, settings.SECRET_KEY,
algorithm=settings.ALGORITHM)
```

Phase 1.7 — Fixed CaseStatusEnum

```
# In app/models/case.py (after split)
class CaseStatusEnum(str, enum.Enum):
    PENDING      = "PENDING"      # Uploaded, not yet processed
    PROCESSING    = "PROCESSING"   # AI inference running
    REVIEWED      = "REVIEWED"     # Doctor has submitted verdict

# ALSO UPDATE your DB if already running:
# ALTER TYPE casestatusenum ADD VALUE 'PROCESSING';
# ALTER TYPE casestatusenum ADD VALUE 'REVIEWED';
# Then migrate existing COMPLETED rows → REVIEWED
```

PHASE 2

Dependencies — RBAC & Auth Guards

Wire the dependency injection system. This is the security backbone.

Phase 2 — Step Checklist

Step	Action	File/Location	Risk
2.1	Create app/dependencies/ directory with __init__.py	app/dependencies/	LOW
2.2	Create app/dependencies/db.py — move get_db from deps.py	app/dependencies/db.py	LOW
2.3	Create app/dependencies/auth.py — move get_current_user, updated to use settings	app/dependencies/auth.py	MED
2.4	Create app/dependencies/rbac.py — WorkspaceContext + require_workspace_role()	app/dependencies/rbac.py	MED
2.5	DELETE app/api/deps.py	DELETE	LOW
2.6	Verify: from app.dependencies.rbac import require_workspace_role	terminal	LOW

Phase 2.2 — db.py

```
# app/dependencies/db.py
from collections.abc import Generator
from sqlalchemy.orm import Session
from app.db.session import SessionLocal

def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Phase 2.3 — auth.py

```
# app/dependencies/auth.py
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTErrror, jwt
from sqlalchemy.orm import Session
from app.core.config import settings
from app.dependencies.db import get_db
from app.models.user import User

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login")
```



```

def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
) -> User:
    exc = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate credentials.",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, settings.SECRET_KEY,
                                algorithms=[settings.ALGORITHM])
        user_id: str | None = payload.get("sub")
        if not user_id:
            raise exc
    except JWTError:
        raise exc
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise exc
    return user

```

Phase 2.4 — rbac.py (Core RBAC Engine)

```

# app/dependencies/rbac.py
from __future__ import annotations
from dataclasses import dataclass
from fastapi import Depends, Header, HTTPException, status
from sqlalchemy.orm import Session
from app.dependencies.auth import get_current_user
from app.dependencies.db import get_db
from app.models.user import User
from app.models.workspace import WorkspaceMember, WorkspaceRoleEnum

_ROLE_RANK: dict[WorkspaceRoleEnum, int] = {
    WorkspaceRoleEnum.DOCTOR: 0,
    WorkspaceRoleEnum.ADMIN: 1,
    WorkspaceRoleEnum.OWNER: 2,
}

@dataclass(frozen=True, slots=True)
class WorkspaceContext:
    workspace_id: str
    member_id: str
    role: WorkspaceRoleEnum
    user: User

def _get_workspace_id(
    x_workspace_id: str | None = Header(None, alias="X-Workspace-Id")
) -> str:
    if not x_workspace_id:
        raise HTTPException(400, "X-Workspace-Id header is required.")
    return x_workspace_id

def require_workspace_role(*allowed_roles: WorkspaceRoleEnum):
    min_rank = min(_ROLE_RANK[r] for r in allowed_roles)

```

```

def _checker(
    workspace_id: str = Depends(_get_workspace_id),
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db),
) -> WorkspaceContext:
    member = (
        db.query(WorkspaceMember)
        .filter(
            WorkspaceMember.user_id == current_user.id,
            WorkspaceMember.workspace_id == workspace_id,
        )
        .first()
    )
    if not member:
        raise HTTPException(status.HTTP_403_FORBIDDEN, "Not a member of this
workspace.")
    caller_rank = _ROLE_RANK.get(WorkspaceRoleEnum(member.role), -1)
    if caller_rank < min_rank:
        raise HTTPException(
            status.HTTP_403_FORBIDDEN,
            f"Requires one of: {[r.value for r in allowed_roles]}."
        )
    return WorkspaceContext(
        workspace_id=workspace_id,
        member_id=member.id,
        role=WorkspaceRoleEnum(member.role),
        user=current_user,
    )
return _checker

```

PHASE 3

Domain Slices — Routers & Services

Migrate each router one at a time. Never break the running server.

The golden rule for this phase: migrate one domain at a time. Keep the old app/api/ router registered until the new domain router is verified to work. Then swap.

Phase 3 — Step Checklist

Step	Action	File/Location	Risk
3.1	Create app/domains/ directory structure with __init__.py in each	app/domains/*/	LOW
3.2	Migrate AUTH domain — router + service + schemas	domains/auth/	MED
3.3	Migrate WORKSPACES domain — fix route ordering bug for /discover	domains/workspaces/	HIGH
3.4	Migrate PATIENTS domain — apply require_workspace_role()	domains/patients/	MED
3.5	Migrate CASES domain — apply role guard + field-level rules in service	domains/cases/	HIGH
3.6	Migrate INVITATIONS domain (extract from workspaces.py)	domains/invitations/	MED
3.7	DELETE entire app/api/ directory	DELETE app/api/	LOW
3.8	Update main.py to register new domain routers	main.py	LOW

Phase 3 — Router Pattern (Cases Example)

```
# app/domains/cases/router.py
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from app.dependencies.db import get_db
from app.dependencies.rbac import WorkspaceContext, require_workspace_role
from app.domains.cases import service
from app.domains.cases.schemas import CaseCreate, CaseResponse, CaseUpdate
from app.models.workspace import WorkspaceRoleEnum

router = APIRouter(prefix="/cases", tags=["cases"])

# — OWNER/ADMIN only —————
@router.post("/", response_model=CaseResponse, status_code=201)
def create_case(
    payload: CaseCreate,
    ctx: WorkspaceContext = Depends(
        require_workspace_role(WorkspaceRoleEnum.ADMIN, WorkspaceRoleEnum.OWNER)
    ),
    db: Session = Depends(get_db),
):
```

```

        return service.create_case(db, payload, ctx)

# — All roles (service enforces DOCTOR field restrictions) —————
@router.put("/{case_id}", response_model=CaseResponse)
def update_case(
    case_id: str,
    payload: CaseUpdate,
    ctx: WorkspaceContext = Depends(
        require_workspace_role(
            WorkspaceRoleEnum.DOCTOR,
            WorkspaceRoleEnum.ADMIN,
            WorkspaceRoleEnum.OWNER,
        )
    ),
    db: Session = Depends(get_db),
):
    return service.update_case(db, case_id, payload, ctx)

# — OWNER/ADMIN only —————
@router.delete("/{case_id}", status_code=204)
def delete_case(
    case_id: str,
    ctx: WorkspaceContext = Depends(
        require_workspace_role(WorkspaceRoleEnum.ADMIN, WorkspaceRoleEnum.OWNER)
    ),
    db: Session = Depends(get_db),
):
    service.delete_case(db, case_id, ctx)

```

Phase 3 — Service Pattern (Cases Example)

```

# app/domains/cases/service.py
from datetime import datetime
from fastapi import HTTPException
from sqlalchemy.orm import Session
from app.dependencies.rbac import WorkspaceContext
from app.domains.cases.schemas import CaseCreate, CaseUpdate
from app.models.case import Case
from app.models.patient import Patient
from app.models.workspace import WorkspaceRoleEnum

_DOCTOR_MUTABLE_FIELDS = frozenset({"verdict", "notes", "status"})

def create_case(db: Session, payload: CaseCreate, ctx: WorkspaceContext) -> Case:
    patient = (
        db.query(Patient)
        .filter(Patient.id == payload.patient_id, Patient.workspace_id ==
ctx.workspace_id)
        .first()
    )
    if not patient:
        raise HTTPException(404, "Patient not found in this workspace.")
    case = Case(**payload.model_dump(), )
    db.add(case)
    db.commit()
    db.refresh(case)

```

```

        return case

def update_case(db: Session, case_id: str, payload: CaseUpdate, ctx:
WorkspaceContext) -> Case:
    case = _get_case_in_workspace(db, case_id, ctx.workspace_id)
    update_data = payload.model_dump(exclude_unset=True)

    if ctx.role == WorkspaceRoleEnum.DOCTOR:
        if case.assigned_to_member_id != ctx.member_id:
            raise HTTPException(403, "Doctors can only update their assigned
cases.")
        disallowed = set(update_data.keys()) - _DOCTOR_MUTABLE_FIELDS
        if disallowed:
            raise HTTPException(403, f"Doctors cannot modify: {disallowed}")

    if "verdict" in update_data:
        update_data["verdict_updated_at"] = datetime.utcnow()

    for field, value in update_data.items():
        setattr(case, field, value)
    db.commit()
    db.refresh(case)
    return case

def _get_case_in_workspace(db: Session, case_id: str, workspace_id: str) -> Case:
    case = db.query(Case).filter(Case.id == case_id).first()
    if not case:
        raise HTTPException(404, "Case not found.")
    if case.patient.workspace_id != workspace_id:
        raise HTTPException(403, "Access denied.")
    return case

```

PHASE 4

Real-Time SSE — Event Bus Integration

Add live updates. Services publish events. Frontend hook auto-updates.

Phase 4 — Step Checklist

Step	Action	File/Location	Risk
4.1	Create app/core/event_bus.py — EventBus class + singleton	app/core/event_bus.py	LOW
4.2	Add GET /workspaces/{id}/events SSE endpoint to workspaces router	domains/workspaces/router.py	MED
4.3	Add event_bus.publish() calls to each service after db.commit()	domains/*/service.py	LOW
4.4	Make service functions async where they publish events	domains/*/service.py	MED
4.5	Create frontend hook: hooks/use-workspace-events.ts	frontend	LOW
4.6	Mount hook in (admin)/layout.tsx and (doctor)/layout.tsx	frontend layout	LOW
4.7	Test: open two browser tabs, invite member in one, verify other updates	manual test	LOW

Phase 4.1 — event_bus.py

```
# app/core/event_bus.py
import asyncio, json
from collections import defaultdict
from dataclasses import dataclass, field
from typing import Any
from uuid import uuid4

@dataclass
class WorkspaceEvent:
    type: str
    payload: dict[str, Any]
    workspace_id: str
    event_id: str = field(default_factory=lambda: str(uuid4()))

    def to_sse(self) -> str:
        data = json.dumps({"type": self.type, "payload": self.payload})
        return f"id: {self.event_id}\nevent: {self.type}\ndata: {data}\n\n"

class EventBus:
    def __init__(self):
        self._subs: dict[str, set[asyncio.Queue]] = defaultdict(set)

    def subscribe(self, workspace_id: str) -> asyncio.Queue:
        q: asyncio.Queue[WorkspaceEvent] = asyncio.Queue(maxsize=100)
```

```

        self._subs[workspace_id].add(q)
        return q

    def unsubscribe(self, workspace_id: str, q: asyncio.Queue) -> None:
        self._subs[workspace_id].discard(q)
        if not self._subs[workspace_id]:
            del self._subs[workspace_id]

    async def publish(self, event: WorkspaceEvent) -> None:
        dead = []
        for q in self._subs.get(event.workspace_id, set()):
            try:
                q.put_nowait(event)
            except asyncio.QueueFull:
                dead.append(q)
        for q in dead:
            self.unsubscribe(event.workspace_id, q)

event_bus = EventBus() # Singleton – import this everywhere

```

Phase 4.2 — SSE Endpoint

```

# Add to app/domains/workspaces/router.py
import asyncio
from fastapi.responses import StreamingResponse
from app.core.event_bus import event_bus

@router.get("/{workspace_id}/events")
async def workspace_event_stream(
    ctx: WorkspaceContext = Depends(
        require_workspace_role(
            WorkspaceRoleEnum.DOCTOR,
            WorkspaceRoleEnum.ADMIN,
            WorkspaceRoleEnum.OWNER,
        )
    ),
) -> StreamingResponse:
    async def stream():
        q = event_bus.subscribe(ctx.workspace_id)
        try:
            yield ": heartbeat\n\n"
            while True:
                try:
                    event = await asyncio.wait_for(q.get(), timeout=30.0)
                    yield event.to_sse()
                except asyncio.TimeoutError:
                    yield ": keepalive\n\n"
        except asyncio.CancelledError:
            pass
    finally:
        event_bus.unsubscribe(ctx.workspace_id, q)

    return StreamingResponse(stream(), media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "X-Accel-Buffering": "no", # Disables Nginx buffering – critical

```

```
        "Connection": "keep-alive",
    })
```

Phase 4.3 — Service Publishing Pattern

```
# Example: invitations/service.py — after accept_invitation DB commit
from app.core.event_bus import event_bus, WorkspaceEvent

async def accept_invitation(
    db: Session, invitation_id: str, current_user: User
) -> dict:
    # ... validation and DB logic ...
    db.commit()

    # Publish AFTER commit — data is now durable
    await event_bus.publish(WorkspaceEvent(
        type="member.joined",
        workspace_id=invitation.workspace_id,
        payload={
            "member": {
                "id": new_membership.id,
                "name": current_user.name,
                "email": current_user.email,
                "role": new_role.value,
            }
        },
    ))
    return {"message": "Invitation accepted"}

# — Event catalogue —————
# member.invited    → invitations/service.py::create_invitation()
# member.joined     → invitations/service.py::accept_invitation()
# member.removed    → workspaces/service.py::remove_member()
# case.created      → cases/service.py::create_case()
# case.updated      → cases/service.py::update_case()
# case.assigned     → cases/service.py::assign_case()
# patient.created   → patients/service.py::create_patient()
```


PHASE 5

Infrastructure — main.py, Alembic, CORS

Final wiring. Clean main.py. Replace Base.metadata.create_all with Alembic.

Phase 5 — Step Checklist

Step	Action	File/Location	Risk
5.1	Rewrite main.py — app factory with lifespan, CORS from settings	<code>main.py</code>	MED
5.2	Install alembic: <code>pip install alembic</code>	<code>requirements.txt</code>	LOW
5.3	Run: <code>alembic init app/db/migrations</code>	<code>terminal</code>	LOW
5.4	Configure <code>alembic.ini</code> to use <code>DATABASE_URL</code> from settings	<code>alembic.ini</code>	MED
5.5	Configure <code>migrations/env.py</code> to import all models for autogenerate	<code>migrations/env.py</code>	MED
5.6	Run: <code>alembic revision --autogenerate -m 'initial schema'</code>	<code>terminal</code>	LOW
5.7	Run: <code>alembic upgrade head</code> — verifies your schema against DB	<code>terminal</code>	HIGH
5.8	Remove <code>Base.metadata.create_all()</code> from main.py startup	<code>main.py</code>	HIGH
5.9	Full smoke test: all endpoints via FastAPI docs (/docs)	<code>manual test</code>	LOW

Phase 5.1 — main.py (Final)

```
# main.py
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.core.config import settings
from app.domains.auth.router import router as auth_router
from app.domains.workspaces.router import router as workspaces_router
from app.domains.patients.router import router as patients_router
from app.domains.cases.router import router as cases_router
from app.domains.invitations.router import router as invitations_router

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup: run DB migrations via Alembic (not create_all)
    yield
    # Shutdown: cleanup if needed

app = FastAPI(
    title="NeuroScan API",
    version="2.0.0",
    lifespan=lifespan,
```

```

        docs_url="/docs" if settings.ENVIRONMENT != "production" else None,
    )

    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.CORS_ORIGINS,
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*", "X-Workspace-Id"],
    )

    app.include_router(auth_router, prefix="/auth")
    app.include_router(workspaces_router, prefix="/workspaces")
    app.include_router(patients_router, prefix="/patients")
    app.include_router(cases_router, prefix="/cases")
    app.include_router(invitations_router, prefix="/invitations")

    @app.get("/health", tags=["system"])
    def health_check() -> dict:
        return {"status": "ok", "version": "2.0.0"}

```

Phase 5.5 — Alembic migrations/env.py

```

# app/db/migrations/env.py (critical section)
from app.core.config import settings
from app.db.base import Base

# Import ALL models so Alembic can detect them for autogenerate
from app.models import (  # noqa: F401
    User, PendingUser,
    Workspace, WorkspaceMember,
    Patient, Case,
    Invitation, JoinRequest,
)

config.set_main_option("sqlalchemy.url", settings.DATABASE_URL)
target_metadata = Base.metadata

```

4. Frontend — useWorkspaceEvents Hook

Mount this hook once in each layout. Every component using React Query with the correct query keys updates automatically — no prop drilling, no polling, no manual refresh.

```
// hooks/use-workspace-events.ts
"use client";
import { useEffect } from "react";
import { useQueryClient } from "@tanstack/react-query";

type EventType =
  | "member.invited" | "member.joined" | "member.removed"
  | "case.created" | "case.updated" | "case.assigned"
  | "patient.created";

const INVALIDATION_MAP: Record<string, (wsId: string) => string[][]> = {
  "member.invited": (id) => [["workspace", id, "invitations"]],
  "member.joined": (id) => [["workspace", id, "members"]],
  "member.removed": (id) => [["workspace", id, "members"]],
  "case.created": (id) => [["workspace", id, "cases"]],
  "case.updated": (id) => [["workspace", id, "cases"]],
  "case.assigned": (id) => [["workspace", id, "cases"]],
  "patient.created": (id) => [["workspace", id, "patients"]],
};

export function useWorkspaceEvents(workspaceId: string | null) {
  const queryClient = useQueryClient();

  useEffect(() => {
    if (!workspaceId) return;
    const url =
      `${process.env.NEXT_PUBLIC_API_URL}/workspaces/${workspaceId}/events`;
    const es = new EventSource(url, { withCredentials: true });

    const handle = (raw: MessageEvent) => {
      const { type } = JSON.parse(raw.data) as { type: EventType };
      const keys = INVALIDATION_MAP[type]?.(workspaceId) ?? [];
      keys.forEach((key) => queryClient.invalidateQueries({ queryKey: key }));
    };

    (Object.keys(INVALIDATION_MAP) as EventType[]).forEach((t) =>
      es.addEventListener(t, handle)
    );
    es.onerror = () => console.warn("[SSE] Reconnecting...");

    return () => {
      (Object.keys(INVALIDATION_MAP) as EventType[]).forEach((t) =>
        es.removeEventListener(t, handle)
      );
      es.close();
    };
  }, [workspaceId, queryClient]);
}

// Usage in layout — one line, affects all children:
// useWorkspaceEvents(currentWorkspaceId);
```


5. Quick Reference — RBAC Decision Matrix

Use this table when implementing any new endpoint to determine which roles to pass to `require_workspace_role()`.

Endpoint	OWNER	ADMIN	DOCTOR	Notes
Create/Delete Workspace	✓	✗	✗	Global ADMIN only
Invite / Remove Member	✓	✓	✗	
View Members	✓	✓	✓	Any member
Create Patient	✓	✓	✗	
View Patient	✓	✓	✓	Any member
Delete Patient	✓	✓	✗	
Create Case	✓	✓	✗	
View Case	✓	✓	✓	Doctor: assigned only
Update Case (verdict/notes)	✓	✓	✓	Doctor: own cases, limited fields
Delete Case	✓	✓	✗	
SSE Event Stream	✓	✓	✓	Any member
Workspace Settings	✓	✗	✗	

6. .env File Template

```
# .env - Copy this to your project root
# Never commit this file to Git

# --- JWT (REQUIRED) ---
SECRET_KEY=generate-with-openssl-rand-hex-32-and-paste-here
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30

# --- Database (Supabase) ---
SUPABASE_DB_USER=postgres.your_project_ref
```

```
SUPABASE_DB_PASSWORD=your_db_password  
SUPABASE_DB_HOST=aws-0-region.pooler.supabase.com  
SUPABASE_DB_PORT=5432  
SUPABASE_DB_NAME=postgres
```

```
# — App —  
ENVIRONMENT=development  
CORS_ORIGINS=["http://localhost:3000"]  
  
# — Generate a strong SECRET_KEY with: —  
# openssl rand -hex 32
```